Logic Building with Loops
Complete Question Bank

8 Phases | Beginner to Intermediate | All Loop Types

Why Loop-Based Logic Matters

Loops are the heartbeat of programming. Whether you are processing data, building games, automating tasks, or training machine learning models — loops are everywhere. Mastering loop logic means you can think systematically, break down complex problems, and write code that is both powerful and efficient.

This question bank is carefully designed to take you from your very first loop all the way to nested logic, mathematical series, and beautiful star patterns. Each problem has been chosen to sharpen a specific skill. Work through them in order, and by the time you finish Phase 8, you will think like a programmer.

How to Use This Guide

1. Work phase by phase — don't skip ahead. Each phase builds on the previous one.
2. Write the code yourself before checking any solution. Struggle is how you learn.
3. After solving each problem, try to optimize it or find an alternate approach.
4. Use a checkmark or highlight each problem you complete — track your progress!
5. Revisit problems you found difficult after completing the full phase.
6. For Additional Practice problems, challenge yourself to solve them without hints.
7. Take breaks! Distributed practice beats cramming every single time.

Phase 1 — While Loop

The while loop is the foundation of all iteration logic. In this phase, you will learn to control repetition using conditions, work with numbers, and solve classic problems like factorials, palindromes, Armstrong numbers, and more. Build your confidence here — this is where it all begins.

Core Problems

1. Print all numbers from 1 to 10 using a loop.
2. Print numbers from 10 down to 1 in reverse order.
3. Print all even numbers between 1 and 100.
4. Print all odd numbers between 1 and 100.
5. Print the multiplication table of a given number from n x 1 to n x 10.
6. Calculate and print the sum of the first n natural numbers.
7. Calculate the sum of all even numbers from 1 up to n.
8. Calculate the sum of all odd numbers from 1 up to n.
9. Calculate and print the factorial of a given number.
10. Find and print the product of all digits of a given number.
11. Count and print the total number of digits in a given number.
12. Reverse the given number and print the reversed value.
13. Check whether the given number is a palindrome.
14. Find and print the sum of digits of the given number.
15. **Check whether the given number is an Armstrong number.**
16. **Check whether the given number is a Perfect number.**
17. Print all prime numbers between 1 and 100.
18. Check whether the given number is a prime number.
19. **Print the Fibonacci series up to n terms.**
20. Find and print the sum of the Fibonacci series up to n terms.
21. Print the square of each number from 1 to n.
22. Print the cube of each number from 1 to n.
23. Print all numbers between a and b divisible by 7.
24. Print all factors of the given number.
25. Find the sum of all factors of the given number.
26. Find HCF of two numbers.
27. Find LCM of two numbers.
28. Find the smallest digit in a number.
29. Find the largest digit in a number.

Additional Practice

30. Count frequency of each digit in a number.
31. Check if a number is a Strong number.
32. Find GCD using the Euclidean method.
33. Check if a number is a Harshad number.

Phase 2 — Do-While Loop

The do-while loop guarantees at least one execution — perfect for input validation, menus, and situations where you need to act before checking a condition. In this phase, you will revisit familiar problems with this powerful loop variant and tackle user-driven programming scenarios.

Core Problems

34. Print numbers from 1 to 10.
35. Print multiplication table of a number.
36. Take input until 0 is entered and print sum.
37. Take input until 0 is entered and print the largest number.
38. Count digits in a number.
39. Reverse a number.
40. Check palindromes.
41. Check Armstrong number.
42. Calculate factorial.
43. **Print Fibonacci series.**
44. Find HCF.
45. **Create a menu-driven program.**
46. Count positive numbers until a negative number is entered.
47. Find sum of digits.
48. Find sum of even and odd digits separately.

Additional Practice

49. Implement a simple calculator using a menu.
50. Validate user password with a retry limit.
51. Keep taking input until a prime number is entered.

Pro Tips — Phases 1 & 2

-> Always trace your loop manually on paper before coding — write out variable values at each step.

-> For digit-based problems, use modulo (%) to extract digits and integer division (/) to shrink the number.

-> The do-while loop is your best friend for menu-driven programs — it ensures the menu shows at least once.

-> Armstrong numbers are a great test for combining loops with power calculations. Try it without built-in power functions!

Phase 3 — For Loop

The for loop is your most structured iteration tool. With a built-in counter, condition, and update all in one line, it's the cleanest way to iterate over a known range. This phase consolidates your understanding by solving all classic problems using this versatile loop.

Core Problems

52. Print numbers from 1 to 10.
53. Print reverse numbers from 10 to 1.
54. Print even numbers (1 to 100).
55. Print odd numbers (1 to 100).
56. Multiplication table.
57. Factorial of a number.
58. **Factorial from 1 to n.**
59. Print primes between 1 to 100.
60. Check the prime number.
61. **Fibonacci series.**
62. Sum of Fibonacci series.
63. Factors of a number.
64. Sum of factors.
65. HCF.
66. LCM.
67. Squares from 1 to n.
68. Cubes from 1 to n.
69. Numbers divisible by 7.
70. Sum of n natural numbers.
71. Sum of even numbers.
72. Sum of odd numbers.

Additional Practice

73. Print Pascal's triangle.
74. Print multiplication table from 1 to 10 (full grid).
75. Find largest and smallest in an array.

Phase 4 — Nested Loop Logic

Nested loops are where logic truly gets interesting. A loop inside a loop lets you work with 2D data, generate patterns, and solve problems that require thinking in rows and columns. Patience and careful tracing will be your most valuable tools here.

Core Problems

76. Print tables from 1 to 10.
77. Print all pairs (i, j).
78. Count factors for each number in a range.
79. Print primes up to n using nested loop.
80. Fibonacci pattern row-wise.
81. **Number triangle pattern.**
82. **Matrix row and column sum.**
83. **Print Pythagorean triplets.**

Additional Practice

84. Print identity matrix.
85. Matrix multiplication.
86. Transpose of a matrix.

Pro Tips — Phases 3 & 4

-> The for loop shines when you know the exact number of iterations upfront — prefer it over while for range-based problems.

-> For nested loops, always trace the outer loop first, then ask: for each outer value, what does the inner loop do?

-> Matrix problems train your ability to think in rows and columns — a critical skill for data science and game development.

-> Pythagorean triplets require three nested loops. Optimize by limiting inner loop ranges using math — brute force works for learning, but think about efficiency too!

Phase 5 — Break & Continue

Sometimes you need to exit a loop early or skip an iteration entirely. Break and continue give you precise control over loop flow. These problems will sharpen your conditional thinking and teach you to write cleaner, more intentional loops.

Core Problems

87. Stop loop when number divisible by 17 appears.
88. Skip numbers divisible by 5.
89. Skip zero input and sum remaining values.
90. **Search a number and break the loop when found.**
91. Stop when a negative number appears.
92. Print only even numbers using continue.
93. **Stop when the running sum exceeds 100.**

Additional Practice

94. Find the first prime number in a given range.
95. Skip multiples of both 3 and 5.
96. Break loop when a palindrome is found.

Phase 6 — Mathematical Series

Series problems combine loops with mathematical patterns. You will compute sums of natural numbers, geometric progressions, harmonic series, factorials, and even Taylor series approximations. This phase bridges programming with mathematics beautifully.

Core Problems

97. Sum of n natural numbers.
98. Sum of n even numbers.
99. Sum of n odd numbers.
100. Arithmetic progression (AP).
101. Geometric progression (GP).
102. Fibonacci series.
103. Sum of Fibonacci series.
104. **Sum of squares (1^2 + 2^2 + 3^2 + ... + n^2).**

105. **Sum of cubes (1^3 + 2^3 + 3^3 + ... + n^3).**
106. Harmonic series (1 + 1/2 + 1/3 + ... + 1/n).
107. Powers of 2 series (1, 2, 4, 8, 16, ...).
108. **Sum of factorial series (1! + 2! + 3! + ... + n!).**
109. Series: 1 + x + x^2 + x^3 + ...
110. **Alternating factorial series.**
111. Strong number check.

Additional Practice

112. Taylor series approximation for sin(x).
113. Taylor series approximation for e^x.
114. Sum of alternating harmonic series (1 - 1/2 + 1/3 - ...).

Pro Tips — Phases 5 & 6

-> Use 'break' sparingly — ask yourself whether a better loop condition could avoid it entirely.

-> Continue is perfect for filtering: process only what matches your criteria, skip everything else.

-> For series problems, always identify the pattern: what changes between terms? That is your loop update logic.

-> Taylor series require precision — use double/float types and test with known values to verify accuracy.

-> The Harmonic series grows slowly — great for exploring floating-point precision concepts.

Phase 7 — Mixed Logical Problems

This is where everything comes together. These problems require you to combine digit analysis, divisibility checks, palindrome detection, and number theory. Working through these builds the kind of flexible, multi-step thinking that real-world programming demands.

Core Problems

115. Find all numbers with an even digit sum between 1 and 100.
116. Count numbers divisible by 7 but not 5 between 1 and 500.
117. **Find all palindromes between 1 and 500.**
118. Find numbers whose digit sum is a multiple of 3.
119. Find binary numbers with an even count of 1s.

120. Print the pattern i x i for values of i from 1 to n.
121. **Find the sum of even digits and odd digits separately.**
122. **Find all Armstrong numbers between 1 and 1000.**
123. **Find all Perfect numbers between 1 and 1000.**
124. Find the number with the maximum digit sum in a range.

Additional Practice

125. Find numbers divisible by both 3 and 5 in a given range.
126. Find numbers with all unique (non-repeating) digits.
127. Find numbers whose reverse is also a prime number.

Phase 8 — Star Pattern Printing

Pattern printing is the ultimate test of nested loop mastery. You must control exactly how many stars appear on each row, how many spaces precede them, and how to build shapes that grow, shrink, or alternate. Each problem below includes the expected output for n = 5 so you know exactly what to aim for.

Core Problems — with Expected Output (n = 5)

128. **Print a single star.**

Output (n = 5):

```
*
```

129. **Print four stars in a row.**

Output (n = 5):

```
* * * *
```

130. **Print n stars in a row.**

Output (n = 5):

```
* * * * *
```

131. **Square pattern of stars.**

Output (n = 5):

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

132. Increasing triangle (left-aligned).

Output (n = 5):

```
*
* *
* * *
* * * *
* * * * *
```

133. Right-aligned triangle pattern.

Output (n = 5):

```
        *
      * *
    * * *
  * * * *
* * * * *
```

134. Even star pattern (stars on even rows).

Output (n = 5):

```
        (row 1: skip)
* *     (row 2)
        (row 3: skip)
* * * * (row 4)
        (row 5: skip)
```

135. Odd star pattern (stars on odd rows).

Output (n = 5):

```
*         (row 1)
          (row 2: skip)
* * *     (row 3)
          (row 4: skip)
* * * * * (row 5)
```

136. Pyramid pattern.

Output (n = 5):

```
      *
     * *
    * * *
   * * * *
  * * * * *
```

137. Alternate star-space pattern.

Output (n = 5):

```
* * * * *
  *   *
* * * * *
  *   *
* * * * *
```

138. Number triangle pattern.

Output (n = 5):

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

139. Repeated number pattern.

Output (n = 5):

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
```

Additional Practice

140. Diamond pattern (grow then shrink).

Output (n = 5):

```
      *
     * *
    * * *
   * * * *
  * * * * *
   * * * *
    * * *
```

```
    * *
     *
```

141. **Hollow square (border only).**

Output (n = 5):

```
* * * * *
*       *
*       *
*       *
* * * * *
```

142. **Hollow pyramid pattern.**

Output (n = 5):

```
     *
    * *
   *   *
  *     *
 *********
```

143. **Pascal's triangle pattern.**

Output (n = 5):

```
     1
    1 1
   1 2 1
  1 3 3 1
 1 4 6 4 1
```

Pro Tips — Phases 7 & 8

-> For mixed logic problems, break each problem into two parts: the outer loop (range), and the inner check (condition).

-> Armstrong numbers require knowing the number of digits first — run a pass to count them, then compute.

-> For star patterns, always plan: outer loop = rows, inner loop 1 = spaces, inner loop 2 = stars. Sketch it on paper!

-> The diamond pattern is just a pyramid + an inverted pyramid. Solve each half separately, then combine.

-> Pattern problems are common in technical interviews — mastering them signals strong loop intuition to employers.